RAJALAKSHMI ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS19P18- DEEP LEARNING CONCEPTS LABORATORY

LAB MANUAL

FINAL YEAR

SEVENTH SEMESTER

2025- 2026

ODD SEMESTER

LIST OF EXPERIMENTS

1. Create a neural network to recognize handwritten digits using MNIST dataset
2. Build a Convolutional Neural Network with Keras/TensorFlow
3. Image Classification on CIFAR-10 Dataset using Convolutional Neural Networks
4. Transfer learning with CNN and Visualization
5. Build a Recurrent Neural Network using Keras/Tensorflow
6. Sentiment Classification of Text using Recurrent Neural Network (RNN)
7. Build autoencoders with Keras/TensorFlow
8. Build GAN with Keras/TensorFlow
9. Perform object detection with YOLO3
10. Mini Project – CNN based or RNN based applications

RAJALAKSHMI ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS19P18- DEEP LEARNING CONCEPTS LABORATORY

LAB PLAN

Sl.No.	Name of the Experiment	Hours Planned
1	Create a neural network to recognize handwritten digits using MNIST dataset	2
2	Build a Convolutional Neural Network with Keras/TensorFlow	2
3	Image Classification on CIFAR-10 Dataset using CNN	2
4	Transfer learning with CNN and Visualization	2
5	Build a Recurrent Neural Network using Keras/Tensorflow	2
6	Sentiment Classification of Text using RNN	2
7	Build autoencoders with Keras/TensorFlow	2
8	Perform object detection with YOLO3	2
9	Build GAN with Keras/TensorFlow	2
10	Mini Project – CNN or RNN based applications	8

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements	Core i5 and above/M1 Chip with minimum 8 GB RAM and 512 GB HDD
Software Requirements	Windows 10/Apple MacOS, Tensorflow, Keras, Numpy, Pandas and Scikit-learn

Course Outcomes (COs)

Course Name: Deep Learning Concepts **Course Code: CS19P18**

Outcome 1	Understand the fundamentals of deep learning based on optimizations and backpropagation and machine learning.
Outcome 2	Train neural network models that converge well without overfitting.
Outcome 3	Learn how to improve the deep learning model performance using error analysis, regularization, hyper parameter tuning.
Outcome 4	Build networks to perform sentiment analysis and work on real-time time series data.
Outcome 5	Analyse different supervised, unsupervised, and reinforcement deep learning models and their applications in real world scenarios; Build, train, test and evaluate neural networks for different applications and data types.

CO-PO –PSO matrices of course

PO/PSO CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
CS19P18.1	3	2	2	-	1	-	-	-	-	-	1	1	2	1	1
CS19P18.2	2	2	2	-	2	-	-	-	-	-	1	2	3	2	2
CS19P18.3	3	3	1	3	2	-	-	-	-	-	1	2	2	2	2
CS19P18.4	2	1	3	-	2	1	1	1	-	1	2	3	3	3	3
CS19P18.5	3	1	1	3	2	2	1	1	1	2	3	3	2	3	3
Average	2.6	1.8	1.8	3.0	1.8	1.5	1.0	1.0	1.0	1.5	1.6	2.2	2.4	2.2	2.2

Note: Enter correlation levels1,2 or 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

INSTALLATION AND CONFIGURATION OF TENSORFLOW

Aim:

To install and configure TensorFlow in anaconda environment in Windows 10.

Procedure:

1. Download Anaconda Navigator and install.

2. Open Anaconda prompt

3. Create a new environment dlc with python 3.7 using the following command:
   conda create -n dlc python=3.7

4. Activate newly created environment dlc using the following command:
   conda activate dlc

5. In dlc prompt, install tensorflow using the following command:
   pip install tensorflow

6. Next install Tensorflow-datasets using the following command:
   pip install tensorflow-datasets

7. Install scikit-learn package using the following command:
   pip install scikit-learn

8. Install pandas package using the following command:
   pip install pandas

9. Lastly, install jupyter notebook
   pip install jupyter notebook

10. Open jupyter notebook by typing the following in dlc prompt:
    jupyter notebook

11. Click create new and then choose python 3 (ipykernel)

12. Give the name to the file

13. Type the code and click Run button to execute (eg. Type import tensorflow and then run)

Useful Learning Resources:

1. https://docs.anaconda.com/free/anaconda/applications/tensorflow/

2. https://conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html#activating-an-environment

EX NO: 1 CREATE A NEURAL NETWORK TO RECOGNIZE HANDWRITTEN DIGITS USING MNIST DATASET

Aim:

To build a handwritten digit's recognition with MNIST dataset.

Procedure:

1. Download and load the MNIST dataset.
2. Perform analysis and preprocessing of the dataset.
3. Build a simple neural network model using Keras/TensorFlow.
4. Compile and fit the model.
5. Perform prediction with the test dataset.
6. Calculate performance metrics.

Useful Learning Resources:

1.https://www.analyticsvidhya.com/blog/2022/07/handwritten-digit-recognition-using-tensorflow/

2. https://www.milindsoorya.com/blog/handwritten-digits-classification

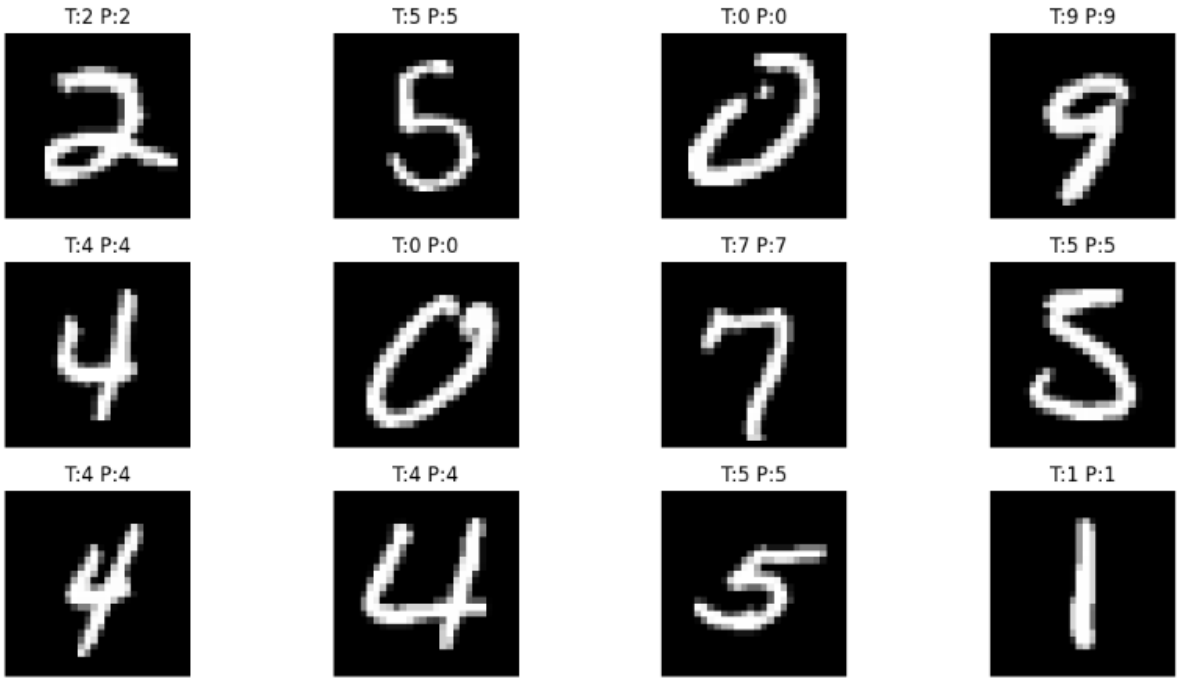
CODE:

```
#Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix
import itertools

# Step 2: Load MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
print("Training data shape:", x_train.shape)  # (60000, 28, 28)
print("Test data shape:", x_test.shape)      # (10000, 28, 28)

# Step 3: Preprocessing
x_train = x_train.reshape(-1, 28*28).astype("float32") / 255.0
x_test = x_test.reshape(-1, 28*28).astype("float32") / 255.0
# Split validation set from training data
x_val = x_train[-10000:]
y_val = y_train[-10000:]
x_train = x_train[:-10000]
y_train = y_train[:-10000]

# Step 4: Build the neural network
model = keras.Sequential([
    layers.Input(shape=(28*28,)),          # input layer (784 features)
    layers.Dense(128, activation='relu'),   # hidden layer 1
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),    # hidden layer 2
    layers.Dropout(0.2),
```

```
    layers.Dense(10, activation='softmax')  # output layer (10 classes)
])
# Step 5: Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Show model summary
model.summary()

# Step 6: Train the model
history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=15,
    batch_size=128,
    verbose=2
)

# Step 7: Evaluate on test data
loss, acc = model.evaluate(x_test, y_test, verbose=2)
print(f"\n✅ Test accuracy: {acc:.4f}")

# Step 8: Make predictions
y_probs = model.predict(x_test)
y_pred = np.argmax(y_probs, axis=1)

# Step 9: Visualize predictions
num_samples = 12
indices = np.random.choice(len(x_test), num_samples, replace=False)
plt.figure(figsize=(12,6))
```

```
for i, idx in enumerate(indices):
    plt.subplot(3,4,i+1)
    plt.imshow(x_test[idx].reshape(28,28), cmap='gray')
    plt.title(f"T:{y_test[idx]} P:{y_pred[idx]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT:



EX NO:2 BUILD A CONVOLUTIONAL NEURAL NETWORK USING KERAS/TENSORFLOW

Aim:

To implement a Convolutional Neural Network (CNN) using Keras/TensorFlow to recognize and classify handwritten digits from the MNIST dataset with high accuracy.

Procedure:

1. Import required libraries (TensorFlow/Keras, NumPy, etc.).
2. Load the MNIST dataset from Keras.
3. Normalize and reshape the image data.
4. Convert labels to one-hot encoded vectors.
5. Build a CNN model with Conv2D, MaxPooling, Flatten, and Dense layers.
6. Compile the model using categorical crossentropy and Adam optimizer.
7. Train the model on training data.
8. Evaluate the model on test data.
9. Display accuracy and predictions.

Useful Learning Resources:

1. https://towardsdatascience.com/build-your-first-cnn-with-tensorflow-a9d7394eaa2e

2.https://www.analyticsvidhya.com/blog/2021/06/building-a-convolutional-neural-network-using-tensorflow-keras/

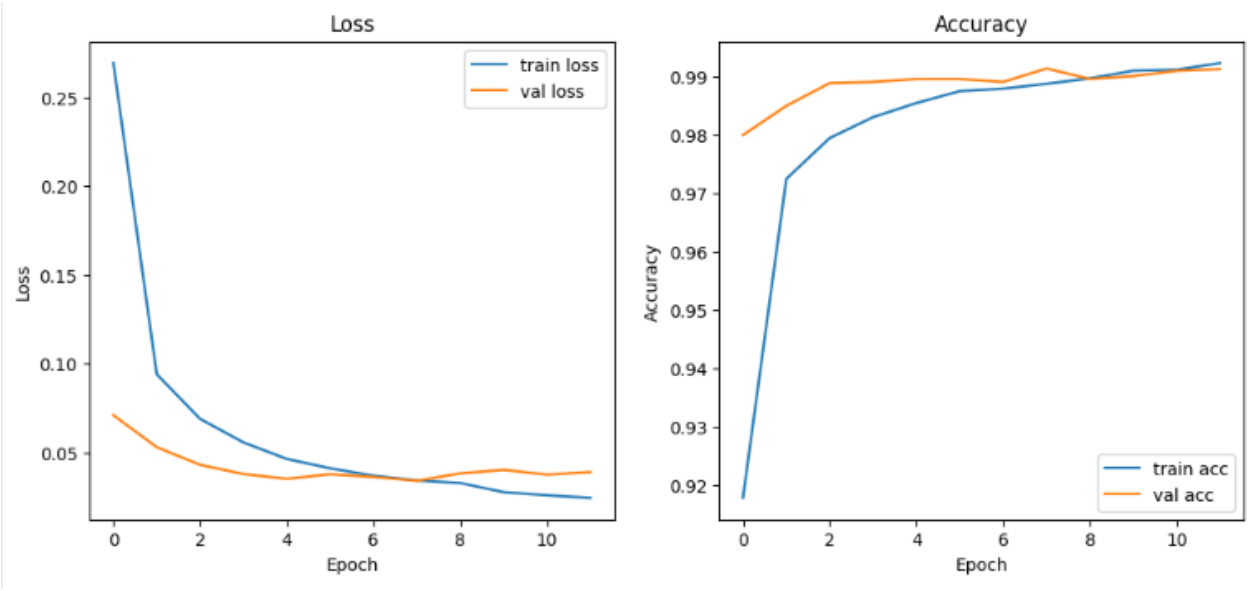
CODE:

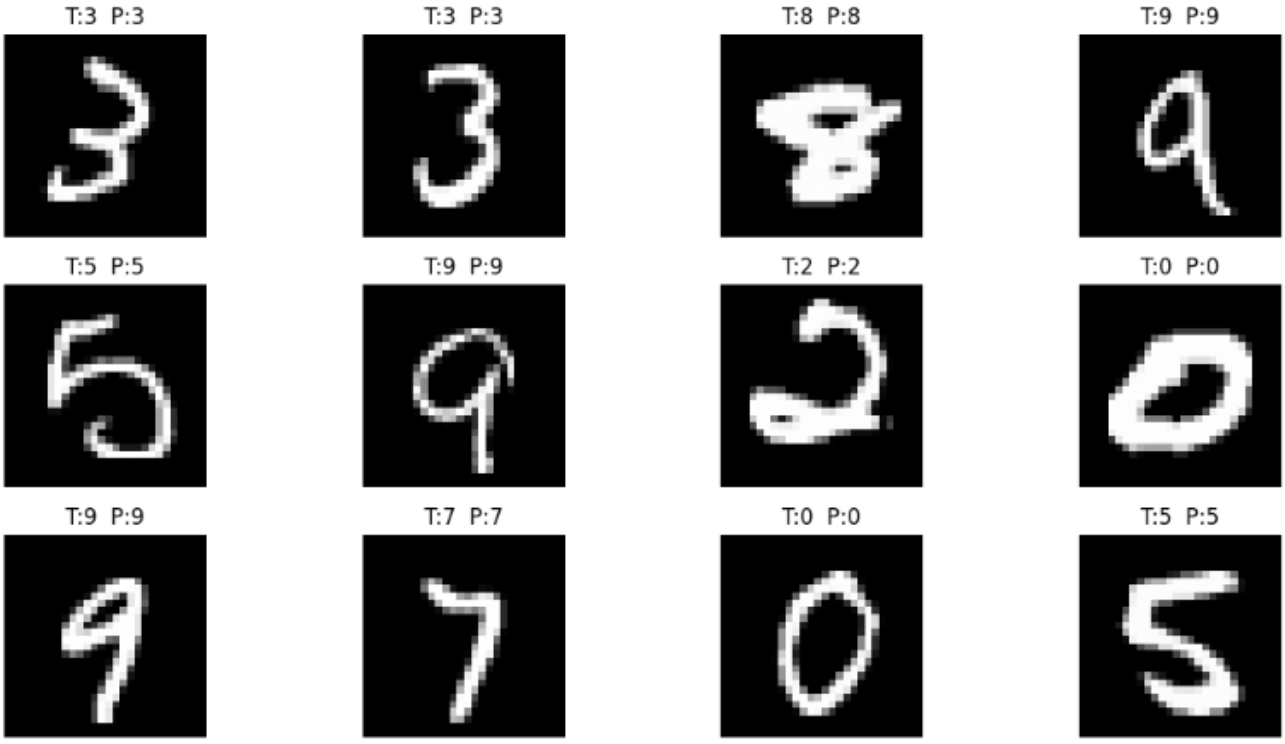
```
# Step 1: Import required libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report, confusion_matrix

# Step 2: Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
print("Raw shapes:", x_train.shape, y_train.shape, x_test.shape, y_test.shape)
# (60000, 28, 28) (60000,) (10000, 28, 28) (10000,)

# Step 3: Normalize and reshape the image data
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32")  / 255.0
# Add channel dimension
x_train = np.expand_dims(x_train, -1)   # shape -> (60000, 28, 28, 1)
x_test  = np.expand_dims(x_test, -1)    # shape -> (10000, 28, 28, 1)

# Step 4: Convert labels to one-hot encoded vectors
num_classes = 10
y_train_cat = keras.utils.to_categorical(y_train, num_classes)
y_test_cat  = keras.utils.to_categorical(y_test,  num_classes)
#Create a validation split (pull 10k from train for validation)
val_size = 10000
x_val = x_train[-val_size:]
y_val = y_train_cat[-val_size:]
x_train = x_train[:-val_size]
y_train_cat = y_train_cat[:-val_size]
```

```
print("Train / Val / Test shapes:", x_train.shape, y_train_cat.shape, x_val.shape, y_val.shape, x_test.shape)
# Step 5: Build CNN model
model = keras.Sequential([
    layers.Conv2D(32, kernel_size=(3,3), activation='relu', input_shape=(28,28,1)),
    layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2,2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])

# Step 6: Compile the model
model.compile(
    optimizer=keras.optimizers.Adam(),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()

# Step 7: Train the model
epochs = 12
batch_size = 128
history = model.fit(
    x_train, y_train_cat,
    validation_data=(x_val, y_val),
    epochs=epochs,
    batch_size=batch_size,
    verbose=2
)
```

```
# Step 8: Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test_cat, verbose=2)
print(f"\nTest loss: {test_loss:.4f} — Test accuracy: {test_acc:.4f}")

# Step 9: Display training accuracy graph
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.title('Loss'); plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.title('Accuracy'); plt.legend()
plt.show()

#Step 10: Make predictions
y_pred_probs = model.predict(x_test)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = y_test

# Step 11: Visualize some sample predictions
n_samples = 12
idxs = np.random.choice(len(x_test), n_samples, replace=False)
plt.figure(figsize=(12,6))
for i, idx in enumerate(idxs):
    plt.subplot(3, 4, i+1)
    plt.imshow(x_test[idx].reshape(28,28), cmap='gray')
    plt.title(f"T:{y_true[idx]}  P:{y_pred[idx]}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

OUTPUT:


Loss
train loss
val loss
Loss
Epoch
Accuracy
train acc
val acc
Accuracy
Epoch


T:3 P:3
T:3 P:3
T:8 P:8
T:9 P:9
T:5 P:5
T:9 P:9
T:2 P:2
T:0 P:0
T:9 P:9
T:7 P:7
T:0 P:0
T:5 P:5

EX NO: 3 IMAGE CLASSIFICATION ON CIFAR-10 DATASET USING CNN

Aim:

To build a Convolutional Neural Network (CNN) model for classifying images from the CIFAR-10 dataset into one of the ten categories such as airplanes, cars, birds, cats, etc.

Procedure:

1. Download and load the CIFAR-10 dataset using Keras/TensorFlow.
2. Visualize and analyze sample images from the dataset.

3, Preprocess the data:

- Normalize the pixel values (divide by 255)
- Convert class labels to one-hot encoded format

4. Build a CNN model using Keras/TensorFlow:
    - Include convolutional, pooling, flatten, and dense layers.
5. Compile the model with suitable loss function and optimizer.
6. Train the model using training data and validate using test data.
7. Evaluate the model using accuracy and loss on test dataset.
8. Perform predictions on new/unseen CIFAR-10 images.

9 Visualize prediction results with sample images and predicted labels.

Useful Learning Resources:

1.https://www.analyticsvidhya.com/blog/2021/01/image-classification-using-convolutional-neural-networks-a-step-by-step-guide/

2. https://www.geeksforgeeks.org/deep-learning/cifar-10-image-classification-in-tensorflow/

CODE:

```
# Step 1: Import required libraries
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np

# Step 2: Download and load the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

# Step 3: Visualize some sample images
class_names = ['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer',
               'Dog', 'Frog', 'Horse', 'Ship', 'Truck']
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_train[i])
    plt.xlabel(class_names[int(y_train[i])])
plt.show()

# Step 4: Preprocess the data
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
# Convert class labels to one-hot encoded format
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)
```

```
# Step 5: Build CNN with BatchNorm + Dropout
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(32, 32, 3)),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

# Step 6: Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
print("y_train shape:", y_train.shape)
```

```
print("y_test shape:", y_test.shape)


# Step 7: Train the model
history = model.fit(x_train, y_train, epochs=20,
                    validation_data=(x_test, y_test),
                    batch_size=64)


# Step 8: Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"\nTest Accuracy: {test_acc:.4f}")
print(f"Test Loss: {test_loss:.4f}")


# Step 9: Perform predictions on test images
predictions = model.predict(x_test)


# Step 10: Visualize predictions
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_test[i])
    predicted_label = np.argmax(predictions[i])
    true_label = np.argmax(y_test[i])
    color = 'green' if predicted_label == true_label else 'red'
    plt.xlabel(f"{class_names[predicted_label]}\n({class_names[true_label]})", color=color)
plt.show()
```

OUTPUT:


Frog
Truck
Truck
Deer
Automobile
Automobile
Bird
Horse
Ship
Cat


Cat
Ship
Ship
Ship
Frog
Frog
(Frog)
Automobile
(Automobile)
Deer
(Frog)
Cat
(Cat)
Truck
(Automobile)

Ex No: 4 TRANSFER LEARNING WITH CNN AND VISUALIZATION

Aim:

To build a convolutional neural network with transfer learning and perform visualization

Procedure:

1. Download and load the dataset.
2. Perform analysis and preprocessing of the dataset.
3. Build a simple neural network model using Keras/TensorFlow.
4. Compile and fit the model.
5. Perform prediction with the test dataset.
6. Calculate performance metrics.

Useful Learning Resources:

1.https://medium.com/analytics-vidhya/car-brand-classification-using-vgg16-transfer-learning-f219a0f09765

2. https://www.kaggle.com/code/kasmithh/transfer-learning-using-keras-vgg-16

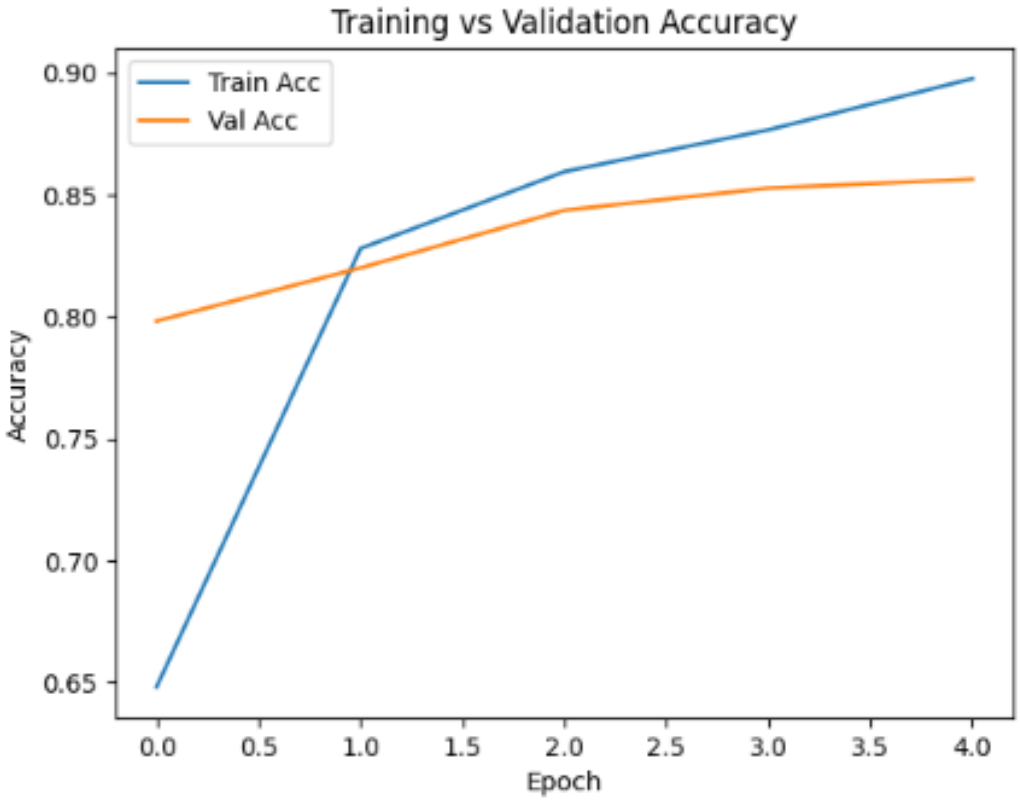
CODE:

```
# Step 1: Import required libraries
import tensorflow as tf
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.preprocessing import image_dataset_from_directory

# Step 2: Load TensorFlow Flowers dataset
dataset, info = tfds.load('tf_flowers', with_info=True, as_supervised=True, shuffle_files=True)
train_dataset = dataset['train']

# Step 3: Split into train, validation, test
train_size = 0.7
val_size = 0.15
test_size = 0.15
train_ds = train_dataset.take(int(info.splits['train'].num_examples * train_size))
val_ds = train_dataset.skip(int(info.splits['train'].num_examples *
train_size)).take(int(info.splits['train'].num_examples * val_size))
test_ds = train_dataset.skip(int(info.splits['train'].num_examples * (train_size + val_size)))

# Step 4: Preprocessing
IMG_SIZE = 160
BATCH_SIZE = 32
def format_example(image, label):
    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
    image = image / 255.0
    return image, label
```

```
train_ds = train_ds.map(format_example).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
val_ds = val_ds.map(format_example).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
test_ds = test_ds.map(format_example).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
class_names = info.features['label'].names
print("Class names:", class_names)

# Step 5: Build Transfer Learning Model
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))
base_model.trainable = False  # freeze base
model = Sequential([
   base_model,
   GlobalAveragePooling2D(),
   Dropout(0.3),
   Dense(len(class_names), activation='softmax')
])
model.compile(optimizer='adam',
          loss='sparse_categorical_crossentropy',
          metrics=['accuracy'])

# Step 6: Train the Model
history = model.fit(train_ds,
                validation_data=val_ds,
                epochs=5)

# Step 7: Evaluate on Test Set
loss, acc = model.evaluate(test_ds)
print(f"Test Accuracy: {acc*100:.2f}%")

# Step 8: Visualization of Training
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training vs Validation Accuracy')
plt.show()

# Step 9: Show One Example per Class with Prediction
plt.figure(figsize=(15, 10))
shown_classes = set()
i = 1
for images, labels in test_ds.unbatch().take(500):  # check first 500 test samples
    label = labels.numpy()
    if label not in shown_classes:
        ax = plt.subplot(2, 3, i)
        plt.imshow(images.numpy())

        # Model prediction
        img_array = tf.expand_dims(images, 0)  # add batch dimension
        predictions = model.predict(img_array, verbose=0)
        pred_label = np.argmax(predictions[0])

        plt.title(f"True: {class_names[label]}\nPred: {class_names[pred_label]}")
        plt.axis("off")

        shown_classes.add(label)
        i += 1
        if len(shown_classes) == len(class_names):
            break
plt.show()
```

OUTPUT:


Training vs Validation Accuracy
Train Acc
Val Acc
Accuracy
Epoch
0.90
0.85
0.80
0.75
0.70
0.65
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0


True: sunflowers
Pred: sunflowers


True: tulips
Pred: tulips


True: roses
Pred: roses


True: daisy
Pred: daisy


True: dandelion
Pred: dandelion

EX NO: 5 BUILD A RECURRENT NEURAL NETWORK (RNN) USING KERAS/TENSORFLOW

Aim:

To build a recurrent neural network with Keras/TensorFlow.

Procedure:

1. Download and load the dataset.
2. Perform analysis and preprocessing of the dataset.
3. Build a simple neural network model using Keras/TensorFlow.
4. Compile and fit the model.
5. Perform prediction with the test dataset.
6. Calculate performance metrics.

Useful Learning Resources:

1. https://machinelearningmastery.com/understanding-simple-recurrent-neural-networks-in-keras/
2. https://victorzhou.com/blog/keras-rnn-tutorial/

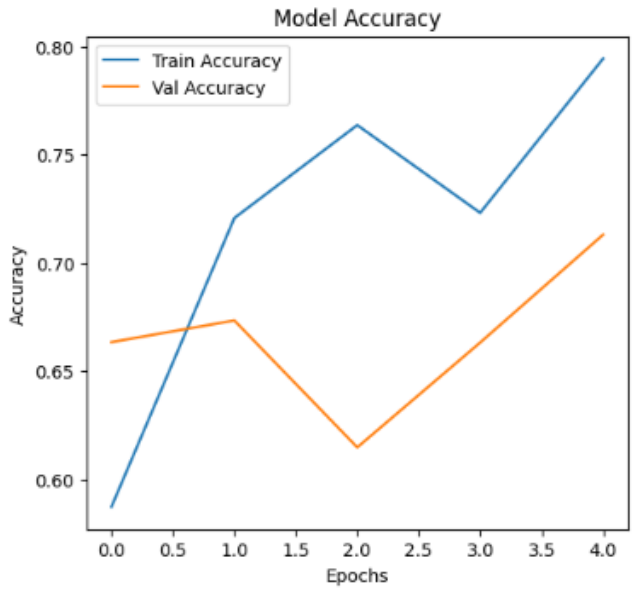
CODE:

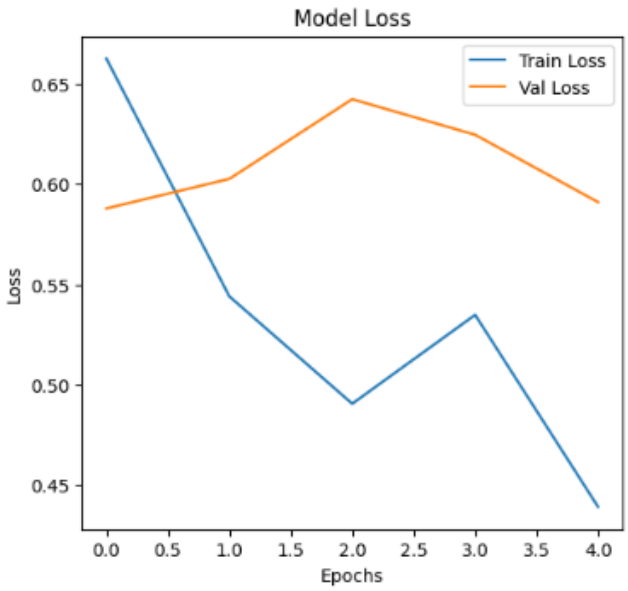
```
# Step 1: Import dependencies
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Sequential
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.metrics import classification_report, accuracy_score

# Step 2: Download and Load Dataset (IMDB Reviews)
# Keep only top 10,000 most frequent words
num_words = 10000
maxlen = 200  # max review length
print("Loading dataset...")
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=num_words)
print(f"Training samples: {len(X_train)}, Test samples: {len(X_test)}")
# Pad sequences to ensure equal length
X_train = pad_sequences(X_train, maxlen=maxlen)
X_test = pad_sequences(X_test, maxlen=maxlen)
print("Data after padding:", X_train.shape, X_test.shape)

# Step 3: Build RNN Model
model = Sequential([
    layers.Embedding(input_dim=num_words, output_dim=128, input_length=maxlen),
    layers.SimpleRNN(128, activation="tanh", return_sequences=False),
    layers.Dense(1, activation="sigmoid")
])
model.summary()

# Step 4: Compile and Train Model
```

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=5,
    batch_size=64,
    verbose=1
)

# Step 5: Perform Predictions
y_pred_probs = model.predict(X_test)
y_pred = (y_pred_probs > 0.5).astype("int32")

# Step 6: Calculate Performance Metrics
acc = accuracy_score(y_test, y_pred)
print("\n ✅ Test Accuracy:", acc)
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=["Negative", "Positive"]))

# Step 7: Visualization of Training
plt.figure(figsize=(12, 5))
# Plot accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train Accuracy")
plt.plot(history.history["val_accuracy"], label="Val Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
```

```
# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Val Loss")
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

OUTPUT:


Model Accuracy
Train Accuracy
Val Accuracy
Accuracy
Epochs
0.80
0.75
0.70
0.65
0.60
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0


Model Loss
Train Loss
Val Loss
Loss
Epochs
0.65
0.60
0.55
0.50
0.45
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

EX NO: 6 SENTIMENT CLASSIFICATION OF TEXT USING RNN

Aim:

To implement a Recurrent Neural Network (RNN) using Keras/TensorFlow for classifying the sentiment of text data (e.g., movie reviews) as positive or negative.

Procedure:

1. Import necessary libraries.
2. Load and preprocess the text dataset (e.g., IMDb).
3. Pad sequences and prepare labels.
4. Build an RNN model with Embedding and SimpleRNN layers.
5. Compile the model with loss and optimizer.
6. Train the model on training data.
7. Evaluate the model on test data.
8. Predict sentiment for new inputs

Useful Learning Resources:

1.https://medium.com/@muhammadluay45/sentiment-analysis-using-recurrent-neural-network-rnn-long-short-term-memory-lstm-and-38d6e670173f

2. https://www.ijert.org/text-classification-using-rnn

CODE:

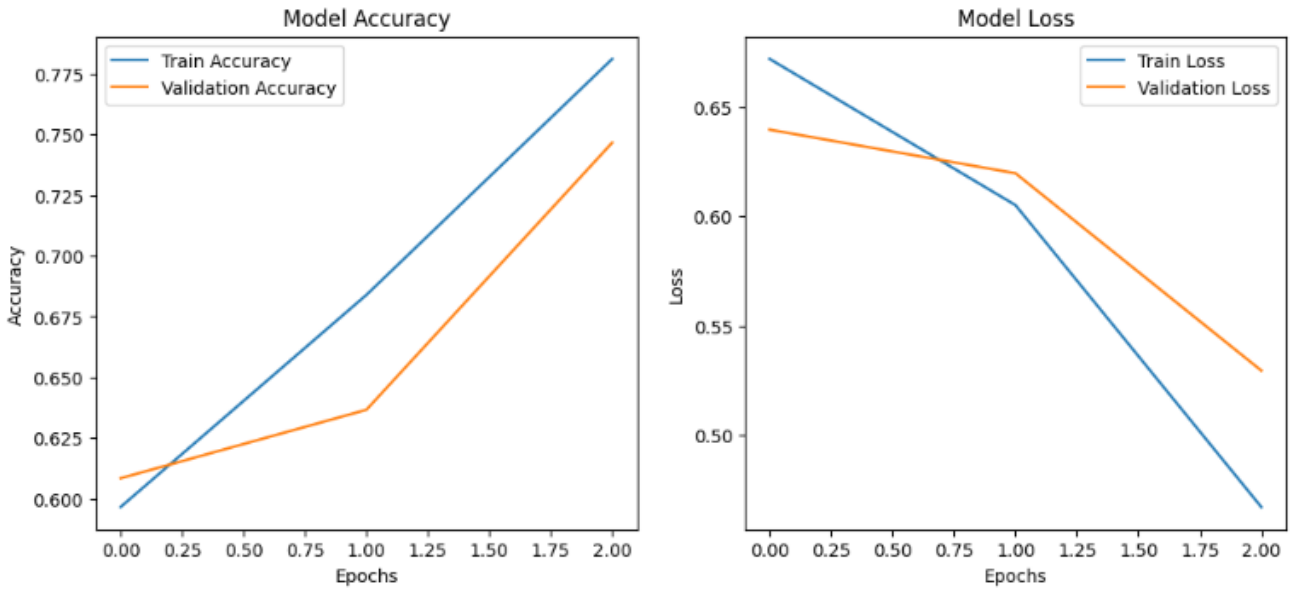
```
# Step 1: Import dependencies
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences
import matplotlib.pyplot as plt

#Steo 2: Load Dataset
max_features = 10000   # Vocabulary size (top 10,000 words)
maxlen = 200          # Maximum review length
print("Loading IMDb dataset...")
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
print(f"Training samples: {len(x_train)}, Test samples: {len(x_test)}")

#Step 3: Preprocess the Dataset
print("Padding sequences...")
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

#Step 4: Build RNN Model
print("Building RNN model...")
model = Sequential()
model.add(Embedding(input_dim=max_features, output_dim=64, input_length=maxlen))
model.add(SimpleRNN(64))   # RNN Layer
model.add(Dense(1, activation='sigmoid'))  # Output Layer

#Step 5: Compile the Model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
```

```
            metrics=['accuracy'])
print(model.summary())

#Step 6: Train the Model
print("Training the model...")
history = model.fit(x_train, y_train,
                    epochs=3,
                    batch_size=64,
                    validation_split=0.2)

#Step 7: Evaluate the Model
print("Evaluating the model...")
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"\nTest Accuracy: {accuracy:.4f}")
print(f"Test Loss: {loss:.4f}")

#Step 8: Predict Sentiment for New Inputs
word_index = imdb.get_word_index()
# Function to decode review back to text
reverse_word_index = {value: key for key, value in word_index.items()}
def decode_review(encoded_review):
    return " ".join([reverse_word_index.get(i - 3, "?") for i in encoded_review])
# Pick one review from test set
sample_review = x_test[1]
decoded = decode_review(sample_review)
print("\nSample Review (decoded):")
print(decoded)
prediction = model.predict(sample_review.reshape(1, -1))[0][0]
print(f"\nPredicted Sentiment Score: {prediction:.4f}")
print("Predicted Label:", "Positive 😊" if prediction > 0.5 else "Negative 😞")
```

```
#Step 9: Plot Training History
plt.figure(figsize=(12,5))

# Accuracy plot
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

# Loss plot
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()

plt.show()
```

OUTPUT:

```
Sample Review (decoded):
psychological ? it's very interesting that robert altman directed this considering the style and structure of his other films still the trademark altman
audio style is evident here and there i think what really makes this film work is the brilliant performance by sandy dennis it's definitely one of her d
arker characters but she plays it so perfectly and convincingly that it's scary michael burns does a good job as the mute young man regular altman playe
r michael murphy has a small part the ? moody set fits the content of the story very well in short this movie is a powerful study of loneliness sexual ?
and desperation be patient ? up the atmosphere and pay attention to the wonderfully written script br br i praise robert altman this is one of his many
films that deals with unconventional fascinating subject matter this film is disturbing but it's sincere and it's sure to ? a strong emotional response
from the viewer if you want to see an unusual film some might even say bizarre this is worth the time br br unfortunately it's very difficult to find in
video stores you may have to buy it off the internet
1/1 [==============================] - 0s 137ms/step

Predicted Sentiment Score: 0.8985
Predicted Label: Positive 😊
```



Ex No: 7 BUILD AUTOENCODERS WITH KERAS/TENSORFLOW

Aim:

To build autoencoders with Keras/TensorFlow.

Procedure:

1. Download and load the dataset.
2. Perform analysis and preprocessing of the dataset.
3. Build a simple neural network model using Keras/TensorFlow.
4. Compile and fit the model.
5. Perform prediction with the test dataset.
6. Calculate performance metrics.

Useful Learning Resources:

1. https://blog.keras.io/building-autoencoders-in-keras.html
2. https://towardsdatascience.com/how-to-make-an-autoencoder-2f2d99cd5103

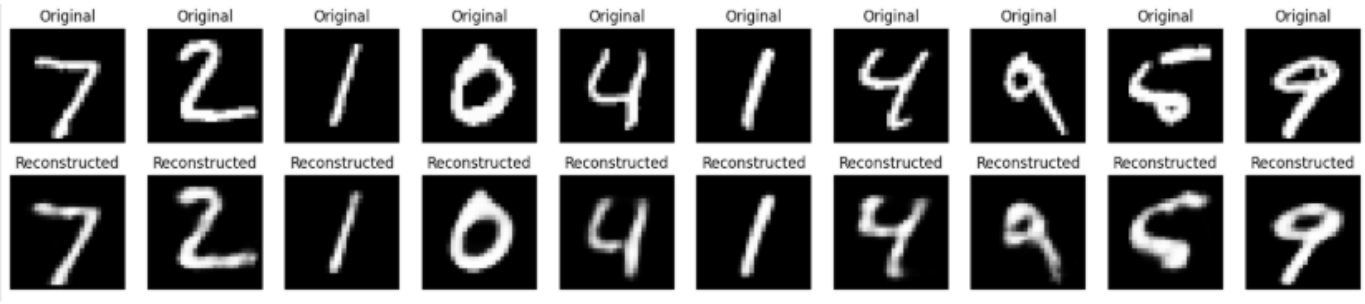
CODE:

```
# Step 1: Import dependencies
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.datasets import mnist

#Steo 2: Load Dataset
(x_train, _), (x_test, _) = mnist.load_data()

#Step 3: Preprocess the Dataset
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
# Flatten 28x28 images into vectors of size 784
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))
print(f"Training data shape: {x_train.shape}, Test data shape: {x_test.shape}")

#Step 4: Build Autoencoder Model
encoding_dim = 32  # Size of encoded representation (compressed feature size)
# Input placeholder
input_img = Input(shape=(784,))
# Encoder network
encoded = Dense(128, activation='relu')(input_img)
encoded = Dense(64, activation='relu')(encoded)
encoded = Dense(encoding_dim, activation='relu')(encoded)
# Decoder network
decoded = Dense(64, activation='relu')(encoded)
decoded = Dense(128, activation='relu')(decoded)
```

```
decoded = Dense(784, activation='sigmoid')(decoded)
# Autoencoder model
autoencoder = Model(input_img, decoded)
# Encoder model (for extracting compressed features)
encoder = Model(input_img, encoded)

#Step 5: Compile the Model
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

#Step 6: Train the Autoencoder
history = autoencoder.fit(x_train, x_train,
                          epochs=20,
                          batch_size=256,
                          shuffle=True,
                          validation_data=(x_test, x_test))

#Step 7: Evaluate the Model
loss = autoencoder.evaluate(x_test, x_test, verbose=0)
print(f"\nTest Reconstruction Loss: {loss:.4f}")

#Step 8: Predict (Reconstruct Images)
decoded_imgs = autoencoder.predict(x_test)

#Step 9: Visualize the Results
n = 10  # Number of digits to display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.title("Original")
    plt.axis("off")
```

```
    # Display reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap="gray")
    plt.title("Reconstructed")
    plt.axis("off")
plt.show()
```

OUTPUT:


Original
Original
Original
Original
Original
Original
Original
Original
Original
Original
Reconstructed
Reconstructed
Reconstructed
Reconstructed
Reconstructed
Reconstructed
Reconstructed
Reconstructed
Reconstructed
Reconstructed

Ex No: 8 **OBJECT DETECTION WITH YOLO3**

Aim:

To build an object detection model with YOLO3 using Keras/TensorFlow.

Procedure:

1. Download and load the dataset.
2. Perform analysis and preprocessing of the dataset.
3. Build a simple neural network model using Keras/TensorFlow.
4. Compile and fit the model.
5. Perform prediction with the test dataset.
6. Calculate performance metrics.

Useful Learning Resources:

1. https://machinelearningmastery.com/how-to-perform-object-detection-with-yolov3-in-keras/
2. https://www.kaggle.com/code/roobansappani/yolo-v3-object-detection-using-keras

CODE:

```
# Step 1: Import required libraries
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt

# Step 2: Load the pretrained YOLOv8 model (nano version for speed)
# This auto-downloads weights from Ultralytics Hub
model = YOLO("yolov3.pt")
print("✅ YOLOv3 model loaded successfully!")

# Step 3: Run object detection on a sample image
img_path = "https://ultralytics.com/images/bus.jpg"
results = model(img_path)

# Step 4: Display results
for r in results:
    annotated_img = r.plot()  # returns an annotated numpy array (BGR)
    # Save the annotated image
    cv2.imwrite("output.jpg", annotated_img)
    # Show inline (Jupyter / Colab)
    plt.imshow(cv2.cvtColor(annotated_img, cv2.COLOR_BGR2RGB))
    plt.axis("off")
    plt.show()

# Step 5: Print detected objects
for r in results:
    for box, cls, conf in zip(r.boxes.xyxy, r.boxes.cls, r.boxes.conf):
        print(f"Detected {model.names[int(cls)]} with confidence {conf:.2f}")

# Step 6: (Optional) Run on webcam
```

```
cap = cv2.VideoCapture(0)  # open webcam
while True:
    ret, frame = cap.read()
    if not ret:
        break
    results = model(frame)
    annotated_frame = results[0].plot()
    cv2.imshow("YOLOv8 Webcam Detection", annotated_frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

\

CODE:

```
# Step 1: Import dependencies
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers

# Step 2: Load and preprocess the dataset (MNIST)
# Load MNIST dataset (28x28 grayscale images)
(X_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
# Normalize images to [-1, 1] for GAN training
X_train = (X_train.astype("float32") - 127.5) / 127.5
X_train = np.expand_dims(X_train, axis=-1)  # Add channel dimension
print("Dataset shape:", X_train.shape)

# Step 3: Build Generator Model
def build_generator(latent_dim):
    model = tf.keras.Sequential([
        layers.Dense(7*7*256, use_bias=False, input_shape=(latent_dim,)),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Reshape((7, 7, 256)),
        layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding="same", use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding="same", use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding="same", use_bias=False, activation="tanh"),
    ])
    return model
```

OUTPUT:

Ex No: 9 BUILD GENERATIVE ADVERSARIAL NEURAL NETWORK

Aim:

To build a generative adversarial neural network using Keras/TensorFlow.

Procedure:

1. Download and load the dataset.
2. Perform analysis and preprocessing of the dataset.
3. Build a simple neural network model using Keras/TensorFlow.
4. Compile and fit the model.
5. Perform prediction with the test dataset.
6. Calculate performance metrics.

Useful Learning Resources:

1. https://pyimagesearch.com/2020/11/16/gans-with-keras-and-tensorflow/

2.https://www.analyticsvidhya.com/blog/2021/06/a-detailed-explanation-of-gan-with-implementation-using-tensorflow-and-keras/

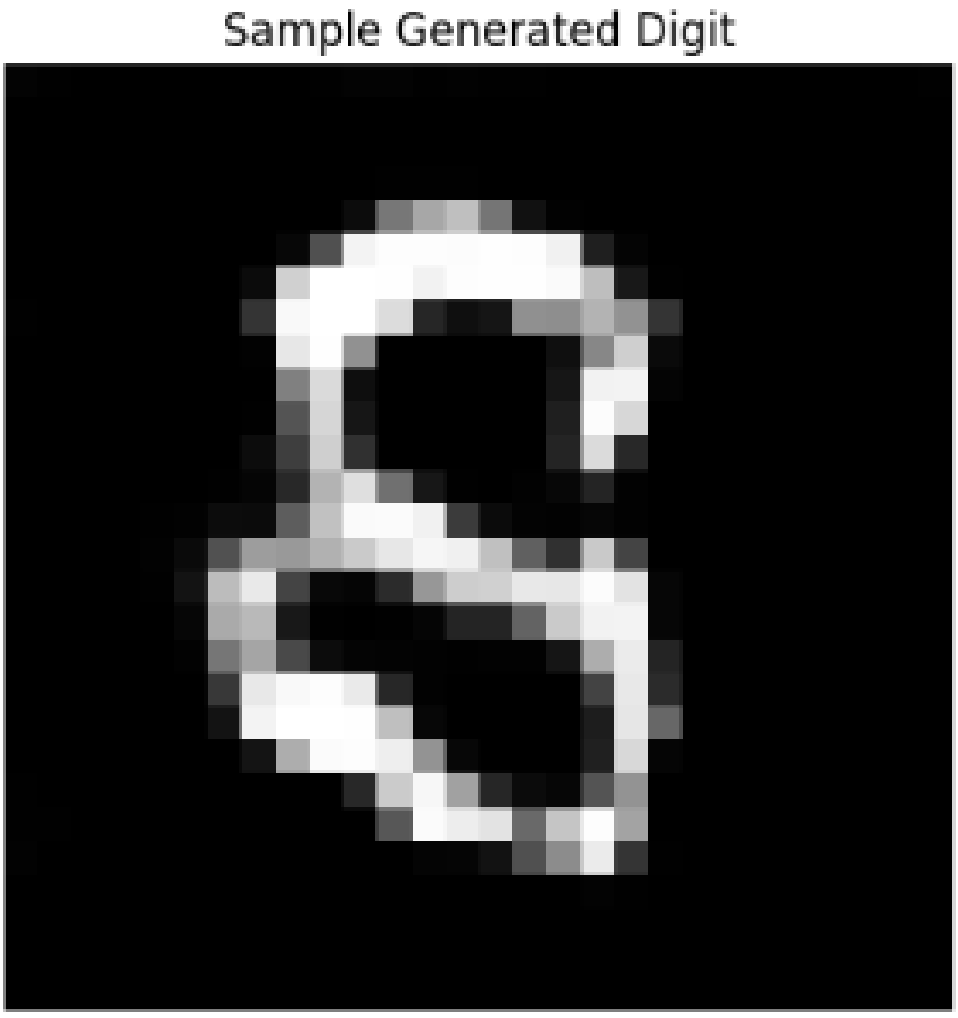
```
# Step 4: Build Discriminator Model
def build_discriminator():
    model = tf.keras.Sequential([
        layers.Conv2D(64, (5, 5), strides=(2, 2), padding="same", input_shape=[28, 28, 1]),
        layers.LeakyReLU(),
        layers.Dropout(0.3),
        layers.Conv2D(128, (5, 5), strides=(2, 2), padding="same"),
        layers.LeakyReLU(),
        layers.Dropout(0.3),
        layers.Flatten(),
        layers.Dense(1, activation="sigmoid"),
    ])
    return model

# Step 5: Compile Models
# Latent space dimension
latent_dim = 100
# Build models
generator = build_generator(latent_dim)
discriminator = build_discriminator()
# Compile discriminator
discriminator.compile(
    loss="binary_crossentropy",
    optimizer=tf.keras.optimizers.Adam(1e-4),
    metrics=["accuracy"]
)
# Build GAN (stacked model)
discriminator.trainable = False  # Freeze discriminator in GAN training
gan_input = tf.keras.Input(shape=(latent_dim,))
fake_image = generator(gan_input)
validity = discriminator(fake_image)
gan = tf.keras.Model(gan_input, validity)
```

```
gan.compile(loss="binary_crossentropy", optimizer=tf.keras.optimizers.Adam(1e-4))


# Step 6: Train GAN
# Training parameters
epochs = 10000
batch_size = 128
sample_interval = 1000
# Labels for real and fake images
real_label = np.ones((batch_size, 1))
fake_label = np.zeros((batch_size, 1))
# Function to save generated images
def save_generated_images(epoch):
    noise = np.random.normal(0, 1, (16, latent_dim))
    gen_imgs = generator.predict(noise)
    gen_imgs = 0.5 * gen_imgs + 0.5  # Rescale to [0, 1]
    fig, axs = plt.subplots(4, 4, figsize=(4, 4))
    count = 0
    for i in range(4):
        for j in range(4):
            axs[i, j].imshow(gen_imgs[count, :, :, 0], cmap="gray")
            axs[i, j].axis("off")
            count += 1
    plt.suptitle(f"Generated Images at Epoch {epoch}")
    plt.show()
# Training loop
for epoch in range(1, epochs + 1):
    # Train Discriminator
    idx = np.random.randint(0, X_train.shape[0], batch_size)
    real_imgs = X_train[idx]
    noise = np.random.normal(0, 1, (batch_size, latent_dim))
    fake_imgs = generator.predict(noise)
```

```
    d_loss_real = discriminator.train_on_batch(real_imgs, real_label)
    d_loss_fake = discriminator.train_on_batch(fake_imgs, fake_label)
    d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)
    # Train Generator
    noise = np.random.normal(0, 1, (batch_size, latent_dim))
    g_loss = gan.train_on_batch(noise, real_label)
    # Print progress
    if epoch % 100 == 0:
        print(f"{epoch} [D loss: {d_loss[0]:.4f}, acc.: {100*d_loss[1]:.2f}] [G loss: {g_loss:.4f}]")
    # Save generated images
    if epoch % sample_interval == 0:
        save_generated_images(epoch)

# Step 7: Evaluate GAN
# Generate some test images
test_noise = np.random.normal(0, 1, (10, latent_dim))
generated_images = generator.predict(test_noise)
print("Generated images shape:", generated_images.shape)
# Show one generated image
plt.imshow(generated_images[0, :, :, 0], cmap="gray")
plt.title("Sample Generated Digit")
plt.axis("off")
plt.show()
```

OUTPUT:

```
1/1 [==============================] - 0s 45ms/step
Generated images shape: (10, 28, 28, 1)
```



LAB VIVA QUESTIONS

1. What is Deep Learning, and how does it differ from traditional machine learning?
2. Explain the concept of neural networks and their role in Deep Learning.
3. What are the main components of a typical neural network?
4. Describe the backpropagation algorithm and its importance in training neural networks.
5. How do you choose the appropriate activation function for a neural network?
6. Discuss the vanishing gradient problem and its impact on Deep Learning.
7. What are some common regularization techniques used in Deep Learning, and how do they prevent overfitting?
8. Explain the concept of convolutional neural networks (CNNs) and their applications.
9. How does pooling (e.g., max pooling) work in CNNs, and what is its purpose?
10. What is data augmentation, and why is it used in CNNs?
11. Discuss the challenges and solutions when working with small datasets in Deep Learning.
12. Describe the architecture and advantages of recurrent neural networks (RNNs).
13. Explain the concept of Long Short-Term Memory (LSTM) cells in RNNs.
14. How do you handle the vanishing gradient problem in RNNs?
15. What is attention mechanism, and how does it improve the performance of sequence-to-sequence models?
16. Discuss the concept of transfer learning and its applications in Deep Learning.
17. How can you fine-tune a pre-trained neural network for a specific task?
18. Explain the differences between supervised, unsupervised, and reinforcement learning in the context of Deep Learning.
19. What are generative adversarial networks (GANs), and how do they work?
20. Describe the main components of a GAN architecture (generator and discriminator).
21. How can GANs be used for image synthesis and style transfer?
22. Discuss the challenges and potential solutions for training GANs.
23. What is the concept of autoencoders, and how are they used in unsupervised learning?
24. Explain the process of dimensionality reduction using autoencoders.
25. How can you use autoencoders for denoising or anomaly detection?
26. Discuss the concept of reinforcement learning and its use in training agents to perform tasks.
27. Explain the role of the reward function in reinforcement learning algorithms.
28. What is Q-learning, and how does it work in reinforcement learning?

29. How can you handle the exploration-exploitation trade-off in reinforcement learning?
30. Describe the challenges and approaches to dealing with high-dimensional action spaces in reinforcement learning.
31. Explain the concept of policy gradients and their advantages in certain reinforcement learning scenarios.
32. Discuss the concept of natural language processing (NLP) and its relation to Deep Learning.
33. How are recurrent neural networks used in natural language processing tasks like language modeling?
34. What is word embedding, and how does it improve the representation of words in NLP models?
35. Explain the architecture and applications of transformer models in NLP.
36. How does the attention mechanism work in transformer-based models like BERT?
37. Discuss the challenges of training large-scale language models and potential solutions.
38. Explain the concept of word2vec and its applications in NLP.
39. What are the differences between CBOW (Continuous Bag of Words) and Skip-gram word2vec models?
40. Describe the process of training a word2vec model.
41. How can word embeddings be visualized and evaluated?
42. Discuss the concept of auto-regressive models in natural language processing.
43. What is beam search, and how does it improve the output generation in sequence-to-sequence models?
44. Explain the concept of self-attention and its use in transformer-based models for NLP.
45. How can you apply transfer learning to pre-trained language models like GPT-3?
46. Discuss the challenges and solutions when working with noisy or unstructured text data in NLP.
47. Explain the concept of style transfer in NLP and its applications.
48. How can you use Deep Learning models for sentiment analysis on text data?
49. Discuss the potential ethical considerations and biases in Deep Learning models for NLP.
50. Describe the process of fine-tuning a pre-trained NLP model for a specific language-related task.